# Complete Express.js Notes (Beginner to Advanced)

## What is Express.js?

Express.js is a minimal and flexible web framework for Node.js used to build APIs, backend servers, authentication systems, and web applications.

## Why Use Express.js?

Express simplifies routing, middleware handling, request parsing, and response management compared to raw Node.js.

## Installing Express

Use npm init -y and npm install express to start a project.

## Creating First Server

```
import express from "express"

const app = express()

app.get("/", (req, res) => {
   res.send("Server Running")
})

app.listen(3000)
```

## HTTP Methods

GET = fetch data, POST = create data, PUT = replace data, PATCH = partial update, DELETE = remove data.

## Route Parameters

```
app.get("/users/:id", (req, res) => {
   console.log(req.params)
})
```

## Query Parameters

```
Example:
/search?name=abhimanyu&age=20
```

## Middleware

Middleware runs between request and response. It can validate data, authenticate users, or modify requests.

## Middleware Syntax

```
const middleware = (req, res, next) => {
   next()
}
```

## express.json()

Converts JSON body into JavaScript object so req.body works properly.

## Routers

Express Router helps separate routes into multiple files for clean architecture.

## MVC Architecture

Model handles database, Controller handles logic, Routes connect endpoints.

## Error Handling

Use try-catch blocks and global error middleware for handling server errors.

## Async Handler

```
const asyncHandler = (fn) => {
   return (req, res, next) => {
      Promise.resolve(fn(req, res, next)).catch(next)
   }
}
```

## JWT Authentication

JWT is used for authentication. Server creates token after login and verifies it on protected routes.

## Cookies

cookie-parser middleware is used for reading cookies from client requests.

## File Uploads

Multer is used to upload files like images and documents.

## Security

Use Helmet, Rate Limiting, Validation, and Password Hashing for backend security.

## Important Packages

express, mongoose, dotenv, cors, bcrypt, jsonwebtoken, multer, helmet, nodemon

## Project Structure

```
src/
■■■ controllers/
■■■ routes/
■■■ models/
■■■ middlewares/
■■■ utils/
■■■ app.js
■■■ index.js
```

## Learning Path

1. Node.js basics → 2. Routing → 3. Middleware → 4. REST APIs → 5. MongoDB → 6. Authentication → 7. Projects

**Final Goal: After learning Express.js properly, you should be able to build REST APIs, authentication systems, secure backend applications, and production-ready servers.**